Index 2: Các loại Index và chiến lược chọn

UNIQUE / PREFIX / INVISIBLE / DESC / COMPOSITE / FUNCTIONAL

Bài giảng Database Management Systems

MySQL 8.x | SQL Indexing & Query Optimization

Tháng 11 năm 2026

Lộ trình và mục tiêu

1. Bối cảnh và integrity
2. Prefix, Invisible và Descending
3. Composite và cấu trúc InnoDB
4. Cardinality và functional indexes
5. Bài tập tình huống

Nội dung phần học

1. Bối cảnh và integrity
2. Prefix, Invisible và Descending
3. Composite và cấu trúc InnoDB
4. Cardinality và functional indexes
5. Bài tập tình huống

Mục tiêu học tập

- Phân biệt các index chuyên biệt và mục đích sử dụng.
- Hiểu leftmost-prefix và cách chọn thứ tự cột cho equality/range/sort.
- Giải thích InnoDB clustered / secondary index, cardinality và selectivity.
- So sánh functional index với query rewrite.
- Phân tích một query thực tế rồi chọn index phù hợp, kiểm chứng bằng `EXPLAIN`.

Điều kiện trước khi thực hành

- Chạy Index 1 để có schema và dữ liệu mẫu.
- Đặc biệt kiểm tra version cho invisible, descending và functional indexes.

```
USE index_lab;
SELECT VERSION() AS mysql_version;
SHOW TABLE STATUS
WHERE Name IN ('customer_search',
               'order_search', 'product_search');
SHOW INDEX FROM order_search;
```

Thận trọng

Tính năng index phụ thuộc MySQL version và storage engine; đừng nhầm cú pháp MySQL với MariaDB.

Bản đồ lựa chọn index

Nhu cầu	Công cụ ứng viên
Dữ liệu bắt buộc unique	UNIQUE index
Chuỗi dài, truy vấn tiền tố	PREFIX index
Kiểm thử bỏ index chưa DROP	INVISIBLE index
Mixed ASC/DESC	DESCENDING composite index
Equality + range trên nhiều cột	COMPOSITE index
Điều kiện theo expression	Rewrite hoặc FUNCTIONAL index

UNIQUE index: ràng buộc nghiệp vụ

- Ngăn hai row cùng giá trị key không-NULL theo quy tắc MySQL.
- `Non_unique = 0` là unique; có thể dùng để lookup.
- Không dùng UNIQUE nếu duplicate là giá trị nghiệp vụ hợp lệ.

```sql
CREATE UNIQUE INDEX uq_customer_email
ON customer_search (email);

SHOW INDEX FROM customer_search
WHERE Key_name = 'uq_customer_email';
```

UNIQUE không giống PRIMARY KEY

Đặc điểm	PRIMARY KEY	UNIQUE
Số lượng / table	Một	Có thể nhiều
Giá trị NULL	Không cho phép	Có thể nhiều NULL*
Vai trò	Định danh chính	Business-key integrity
InnoDB clustered	Thông thường có	Thường secondary

Điểm cần nhớ

*Trong MySQL, với cột nullable, UNIQUE cho phép nhiều NULL; nếu email bắt buộc, thêm `NOT NULL`.

Thử trùng email: điều gì xảy ra?

- Nếu email đã tồn tại, MySQL trả lỗi duplicate key.
- Nếu schema cho `email = NULL`, nhiều hàng NULL có thể vẫn hợp lệ.

```
-- Thu nghiem tren schema lab
INSERT INTO customer_search
(customer_id, first_name, last_name,
 email, country, credit_limit)
VALUES (999001, 'Test', 'Duplicate',
        'customer103@example.test', 'USA', 1000);
```

Thận trọng

Không kết luận dữ liệu duplicate chỉ bằng một giá trị thử; kiểm tra index được tạo thành công trước.

Nội dung phần học

1. Bối cảnh và integrity
2. Prefix, Invisible và Descending
3. Composite và cấu trúc InnoDB
4. Cardinality và functional indexes
5. Bài tập tình huống

PREFIX index: index một phần của chuỗi

- Giảm lượng ký tự lưu trong key (hữu ích với chuỗi dài).
- Prefix ngắn có thể tạo nhiều trùng lặp và phải kiểm tra thêm row.
- `LIKE '%example'` không phải dạng prefix-search phù hợp.

```sql
CREATE INDEX idx_customer_email_prefix
ON customer_search (email(12));

EXPLAIN SELECT customer_id, email
FROM customer_search
WHERE email LIKE 'customer10%';
```

Chọn độ dài prefix bằng dữ liệu

- So sánh số distinct prefix với full values.
- Kết hợp với kích thước index, charset và pattern truy vấn.

```
SELECT COUNT(*) AS n,
  COUNT(DISTINCT LEFT(email,4)) AS d4,
  COUNT(DISTINCT LEFT(email,8)) AS d8,
  COUNT(DISTINCT LEFT(email,12)) AS d12,
  COUNT(DISTINCT email) AS full_d
FROM customer_search;
```

Thận trọng

UNIQUE(email(12)) chỉ bảo đảm tính duy nhất của 12 ký tự đầu; thường không phù hợp để đảm bảo email đầy đủ là unique.

PREFIX index: minh họa va chạm

- Hai chuỗi khác nhau có thể chung prefix, làm giảm selectivity.
- Tăng prefix length cải thiện phân biệt nhưng thường làm index lớn hơn.

Email	Prefix 8
customer101@a.test	customer
customer102@b.test	customer
nguyen.an@example	nguyen.a

Điểm cần nhớ

Prefix là trade-off không gian/lọc ứng viên; không đồng nghĩa full-text indexing.

INVISIBLE index: thử nghiệm bỏ index

- Invisible index vẫn tồn tại và vẫn được cập nhật khi DML.
- Optimizer mặc định bỏ qua nó khi lập plan.
- Có thể thử ảnh hưởng trước khi quyết định DROP.

```
ALTER TABLE customer_search
ALTER INDEX idx_customer_country INVISIBLE;

EXPLAIN SELECT customer_id FROM customer_search
WHERE country = 'USA';

ALTER TABLE customer_search
ALTER INDEX idx_customer_country VISIBLE;
```

Invisible: những ngoại lệ quan trọng

- Primary key không được invisible.
- UNIQUE invisible index vẫn **enforce uniqueness**.
- Invisible index vẫn tốn storage và chi phí update.
- Một số unique index dùng làm clustered key ngầm không thể invisible.
- Không đổi cấu hình optimizer toàn cục chỉ để thử một query.

Điểm cần nhớ

Quy trình: đo baseline → invisible → chạy tests → visible lại hoặc drop khi đã xác minh.

DESCENDING index cho ORDER BY phức hợp

- Mixed direction là tình huống điển hình cần quan tâm.
- Một B-tree ASC một cột có thể được duyệt ngược; không phải cứ `DESC` là cần index mới.

```
CREATE INDEX idx_order_status_date_desc
ON order_search (status ASC, order_date DESC);

EXPLAIN SELECT order_number, status, order_date
FROM order_search
ORDER BY status ASC, order_date DESC;
```

Ví dụ tại sao chỉ ORDER BY DESC chưa đủ

- Kiểm tra `Extra` xem có `Using filesort` hay không.
- Khi filter/join/ORDER BY khác pattern, optimizer vẫn có thể chọn sort.

```
-- Index thuong
CREATE INDEX idx_order_date
ON order_search (order_date);

-- Co the duyet nguoc index theo date
EXPLAIN SELECT order_number, order_date
FROM order_search
ORDER BY order_date DESC LIMIT 10;
```

Điểm cần nhớ

Index direction chỉ là một thành phần; key order và predicates quyết định ích lợi thực tế.

Nội dung phần học

1. Bối cảnh và integrity
2. Prefix, Invisible và Descending
3. Composite và cấu trúc InnoDB
4. Cardinality và functional indexes
5. Bài tập tình huống

Composite index là chuỗi key có thứ tự

- Keys sắp theo `country` trước, sau đó `credit_limit` trong từng country.
- Đây là một index gồm hai key parts, không phải hai index một cột.

```sql
CREATE INDEX idx_customer_country_credit
ON customer_search (country, credit_limit);

SELECT customer_id, credit_limit
FROM customer_search
WHERE country = 'USA'
  AND credit_limit >= 50000;
```

Leftmost prefix: trường hợp phù hợp / không

Predicate	**Với** `(country, credit_limit)`
`country='USA'`	Thường phù hợp
`country='USA' AND credit_limit>=50000`	Thường rất phù hợp
`credit_limit>=50000`	Không có leftmost `country`
`country IN (...) AND credit_limit>?`	Tùy phạm vi & optimizer

Điểm cần nhớ

Từ MySQL 8, skip scan có thể giúp một số truy vấn thiếu cột trái; không thay thế nguyên tắc thiết kế theo workload.

Equality trước, range tiếp theo

- Equality cố định dải `customer_id`.
- Range của `order_date` được áp dụng trong dải đó.
- Sort theo date có thể tận dụng thứ tự key khi điều kiện phù hợp.

```sql
SELECT order_number, order_date, status
FROM order_search
WHERE customer_id = 103
  AND order_date >= '2004-01-01'
  AND order_date < '2005-01-01'
ORDER BY order_date DESC;

CREATE INDEX idx_order_customer_date
ON order_search(customer_id, order_date);
```

Quy tắc chọn thứ tự cột không chỉ cardinality

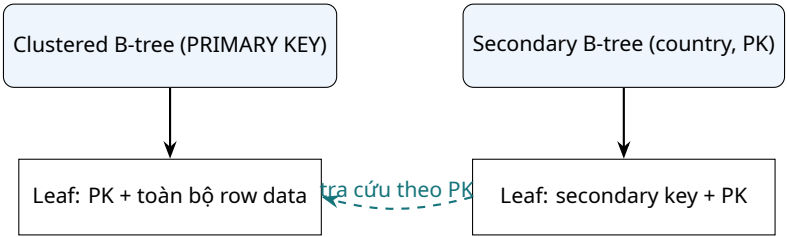
- Cột equality trong query quan trọng thường là ứng viên đầu.
- Cột range/sort tiếp theo cần xét cùng với thứ tự và tần suất query.
- Kiểm tra index đã có: có leftmost prefix bao phủ chưa?
- Tổng số row được lấy và nhu cầu covering index cũng quan trọng.
- Chi phí INSERT/UPDATE/DELETE có thể khiến index mới không đáng giá.

Điểm cần nhớ

Không có một quy tắc “cardinality lớn nhất luôn phải đứng đầu” cho mọi query.

Clustered index trong InnoDB

- Một table InnoDB có một clustered index chứa dữ liệu row.
- Secondary index mang theo PK để tra cứu row khi cần cột ngoài index.



InnoDB chọn clustered key thế nào?

1. Dùng PRIMARY KEY nếu khai báo.
2. Nếu không có PRIMARY KEY, dùng unique index đầu tiên mà tất cả key parts NOT NULL (khi phù hợp).
3. Nếu không có, InnoDB tạo clustered key ẩn trên row ID.

Điểm cần nhớ

Nên có primary key rõ ràng, ngắn, ổn định; PK dài làm các secondary index rộng hơn.

Covering index và index-only lookup

- Nếu tất cả cột cần thiết có thể lấy từ index, tránh một số lần truy cập row.
- Với InnoDB, secondary index cũng chứa primary key.
- `Extra = Using index` thường chỉ tình huống covering index (khác `Using index condition`).

```
CREATE INDEX idx_order_customer_date
ON order_search(customer_id, order_date);

EXPLAIN SELECT customer_id, order_date
FROM order_search
WHERE customer_id = 103;
```

Nội dung phần học

1. Bối cảnh và integrity
2. Prefix, Invisible và Descending
3. Composite và cấu trúc InnoDB
4. Cardinality và functional indexes
5. Bài tập tình huống

Cardinality không phải số distinct tuyệt đối

- `SHOW INDEX.Cardinality` là estimate, có thể khác phép đếm exact.
- Index trên `status` có thể ít distinct; `order_number` unique có nhiều distinct.
- Selectivity còn phụ thuộc tần suất từng value, không chỉ số distinct.

```sql
SHOW INDEX FROM order_search;

SELECT COUNT(DISTINCT customer_id) AS exact_distinct
FROM order_search;
```

Selectivity: trường hợp lệch phân phối

Giả sử bảng 1.000.000 đơn, 95% có `status = Shipped`.

- Index `(status)` có thể ít hữu ích khi truy vấn chọn 950.000 rows.
- Cùng index vẫn có thể hữu ích với trạng thái hiếm chiếm 0,1%.
- Statistics trung bình không luôn mô tả skew của một giá trị.
- Thử nhiều filter values; quan sát `rows`, actual timing và workload.

Điểm cần nhớ

Index nào cũng cần được đánh giá trong ngữ cảnh phân phối dữ liệu.

Functional index: khi predicate chứa function

- Function trên indexed column có thể cản trở range scan thông thường.
- Đừng vội tạo functional index; thử rewrite theo interval.

```
CREATE INDEX idx_order_date
ON order_search(order_date);

EXPLAIN SELECT COUNT(*)
FROM order_search
WHERE YEAR(order_date) = 2004;
```

Rewrite YEAR(date) bằng khoảng nửa mở

- Giữ điều kiện trên cột gốc, thường index-friendly hơn.
- Khoảng nửa mở tránh nhầm về thời gian cuối ngày với DATETIME/TIMESTAMP.
- Dùng `EXPLAIN` cho hai phiên bản; so sánh tính đúng đắn và performance.

```sql
-- Thay YEAR(order_date) = 2004
SELECT COUNT(*) FROM order_search
WHERE order_date >= '2004-01-01'
  AND order_date < '2005-01-01';
```

Functional index: cú pháp và giới hạn

- Cú pháp functional key part cần cặp dấu ngoặc quanh expression.
- Có giới hạn về loại expression và phiên bản MySQL.
- Chỉ có ý nghĩa khi workload thực sự thường xuyên truy vấn theo expression đó.

```
CREATE INDEX idx_order_year
ON order_search ((YEAR(order_date)));

EXPLAIN SELECT COUNT(*) FROM order_search
WHERE YEAR(order_date) = 2004;
```

Thận trọng

Không dùng làm cách thay thế mặc định cho query rewrite đơn giản, có thể bảo trì tốt hơn.

Bảng so sánh các giải pháp

Tình huống	Giải pháp ưu tiên	Kiểm chứng
Email phải distinct	UNIQUE(email)	Insert duplicate
Email search prefix	PREFIX(email(n))	Distinct prefixes
Test bỏ index	INVISIBLE	EXPLAIN, workload
ASC / DESC trộn	DESC composite	Extra / filesort
customer + date	COMPOSITE	Key, rows, timing
YEAR(date)	Range rewrite trước	Plan + semantics

Nội dung phần học

1. Bối cảnh và integrity

2. Prefix, Invisible và Descending

3. Composite và cấu trúc InnoDB

4. Cardinality và functional indexes

5. Bài tập tình huống

Case 1: tìm đơn theo khách hàng

Workload A: 60% query lọc customer; 25% query lọc customer + date; 15% lọc status.

1. Đề xuất 1–2 index ứng viên (đừng tạo tất cả).
2. Phân tích leftmost prefix và chi phí write.
3. Chạy EXPLAIN và kiểm tra Index 1 metadata.

```sql
SELECT order_number, status, order_date
FROM order_search
WHERE customer_id = 103
  AND order_date >= '2004-01-01'
ORDER BY order_date DESC;
```

Điểm cần nhớ

Đáp án ứng viên: `(customer_id, order_date)`; index `(customer_id)` có thể trở nên redundant tùy workload.

Case 2: email dài và unique

Bài toán: email tối đa 300 ký tự, mỗi người có thể chưa cung cấp email.

- Nếu business rule không cho email trùng nhưng cho nhiều người chưa nhập, dùng UNIQUE full email và nullable.
- Nếu query hay dùng prefix search, đo prefix selectivity trước khi thêm index phụ.
- Không dùng UNIQUE(prefix) để enforce email đầy đủ.
- Đánh giá collation: tính duy nhất có thể phụ thuộc quy tắc so sánh chuỗi.

Case 3: tối ưu sort danh sách đơn

- So sánh với `(status, order_date)` thông thường.
- `LIMIT` có thể làm việc duyệt index theo order hấp dẫn hơn full sort.
- Không khẳng định có hoặc không có filesort nếu chưa xem EXPLAIN.

```sql
SELECT order_number, status, order_date
FROM order_search
ORDER BY status ASC, order_date DESC
LIMIT 50;

-- Ung vien
CREATE INDEX idx_status_date_desc
ON order_search(status ASC, order_date DESC);
```

Bài tập cuối bài: chẩn đoán và thiết kế

- Với 3 query trên, xác định index nào có thể reuse và index nào redundant.
- Đề xuất một tình huống dùng invisible index trước khi drop.
- So sánh `Cardinality` với `COUNT(DISTINCT ...)`.
- Viết hai phiên bản query `YEAR(order_date)=2004` và range rewrite.
- Giải thích rõ tác động của primary-key length lên secondary indexes.

Tổng kết Index 2

- **UNIQUE:** bảo đảm integrity, không đơn thuần tăng tốc.
- **PREFIX/INVISIBLE/DESC:** công cụ chuyên biệt cho search, test, sort.
- **COMPOSITE:** thứ tự key parts phải khớp query patterns.
- **InnoDB:** clustered leaf chứa row, secondary leaf mang PK.
- **FUNCTIONAL:** cân nhắc khi rewrite không phù hợp; đo bằng EXPLAIN.

Điểm cần nhớ

Tham khảo: vudmvn.github.io/dbms-course → Index Optimization → Index 2.